APPENDIX

Appendix: Leftovers, Corrections Log & Revision Plan

Small topics from the original notes not covered elsewhere, a consolidated log of every correction made to the original notes, and a suggested way to revise this book before interviews

IN THIS PHASE

A.1 Leftover Topics from the Original Notes
A.2 Corrections Log
A.3 How to Revise This Book

A.1 Leftover Topics from the Original Notes

Nested interfaces

An interface declared inside a class or another interface. It is implicitly `static`, and it groups a contract with its owner (like `Map.Entry`).

```
interface Bank {
    interface Rules { int MAX_TXN = 200_000; }      // nested interface: implicitly static
    void open();
}
class RbiPolicy implements Bank.Rules { }          // implement as Outer.Inner

class Card {
    public interface Validator { boolean ok(String number); }
}
Card.Validator v = n -> n.length() == 16;          // functional, so a lambda works
System.out.println(Bank.Rules.MAX_TXN);            // 200000
```

SequenceInputStream: read several streams as one

```
try (SequenceInputStream sis = new SequenceInputStream(
        new FileInputStream("header.txt"), new FileInputStream("body.txt"))) {
    System.out.print(new String(sis.readAllBytes()));   // header content, then body content
}
```

It is useful for concatenating file parts. With NIO.2 you would more often write the parts to one `OutputStream`, or use `Files.write` with `StandardOpenOption.APPEND`.

FileDescriptor: the OS-level handle

A `FileDescriptor` is **not** a file or a stream. It is the low-level handle the operating system uses for an open file, socket or device.

Member	Meaning
`FileDescriptor.in` / `out` / `err`	Standard input, output and error (what `System.in/out/err` wrap)
`stream.getFD()`	Handle of an open `FileInputStream`/`FileOutputStream`/`RandomAccessFile`
`fd.sync()`	Force buffered OS data to physical storage (durability for critical writes)
`fd.valid()`	Whether the handle is still open

```
try (FileOutputStream fos = new FileOutputStream("ledger.dat")) {
    fos.write(bytes);
    fos.getFD().sync();        // make sure the ledger entry really hit the disk
}
```

FilePermission

`java.io.FilePermission` represents access rights (`read`, `write`, `execute`, `delete`, `readlink`) on a path. A path ending in `-` means "recursively everything below"; `*` means "files directly in the folder".

```
FilePermission rw = new FilePermission("/data/-", "read,write");
rw.implies(new FilePermission("/data/reports/a.txt", "read"));   // true
rw.implies(new FilePermission("/etc/passwd", "read"));           // false
```

> **NOTE**
> Permissions were enforced by the `SecurityManager`, which is **deprecated for removal since Java 17** (JEP 411) and permanently disabled in Java 24. Today, file access is controlled by OS permissions, containers and application-level checks, so `FilePermission` is mostly of historical interest.

URL-related exceptions

Exception	Cause	Example
`URISyntaxException` (checked)	Invalid URI syntax	`new URI("http://bad host")` (space in host)
`MalformedURLException` (checked, an `IOException`)	Unknown protocol or bad URL form	`URI.create("http://abc").toURL()` → "unknown protocol: http"
`IllegalArgumentException`	`URI.create` with bad syntax, or `toURL()` on a relative URI	`URI.create("relative/path").toURL()` → "URI is not absolute"

A.2 Corrections Log

Every statement from the two original note files that was corrected in this book, in one place, so you don't carry a wrong idea into an interview.

Phase	Original notes said	Correct statement
1	Bytecode is secure because viruses attack object code, not bytecode	Security comes from the bytecode verifier, no pointer arithmetic, class-loader isolation and runtime checks
1	Distributed means code need not run sequentially	That describes multithreading. Distributed means built-in networking support.
1	Java is an interpreted language	Java is compiled (to bytecode) **and** interpreted/JIT-compiled at runtime
2	`int MIN_AGE = 21;` is a constant	Only `final` makes a constant; capital letters are just a convention
2	float/double hold 6/15 "decimal places"	About 6–7 / 15–16 **significant digits** in total
2	`boolean` occupies 1 bit	Its size is JVM-dependent (usually 1 byte as a field)
2	`char` stores an ASCII value	It stores a UTF-16 code unit (Unicode); ASCII is just the first 128 values
2	53 keywords, including `null`	50 keywords + _; `true`, `false`, `null` are **literals**
2	`.` is a relational operator	`.` is the member-access operator
3	`switch (2.1f)` with float cases works	float, double, long and boolean are never allowed in a switch

Phase	Original notes said	Correct statement
3	`switch ('a')` matches `case 1`	`'a'` is 97, so nothing matches
4	"Why Strings are mutable"	Strings are **immutable**; reassigning a variable doesn't change the shared object
4	Pattern and Matcher method descriptions	They were swapped. `compile`/`matcher`/`split`/`quote` belong to Pattern; `find`/`matches`/`group` to Matcher.
5	`hashCode()` is a unique id or memory address	Not unique, and not an address: just a number for hash-based collections
5	`return` must be the last statement	Several returns are fine; only code after a return in the same block is unreachable
6	`strictfp` gives platform-independent results	Obsolete since Java 17: all floating point is strict by default
6	No multiple inheritance makes Java "partial OOP"	Primitives and static members are why Java isn't *purely* OOP
6	Removing `super()` breaks compilation	The compiler inserts `super()`; it fails only if the parent lacks a no-arg constructor
7	Final classes are immutable	`final` only prevents subclassing; immutability needs private final fields, no setters and defensive copies
7	Since Java 8 there is no need for abstract classes	Interfaces still can't hold instance state or constructors; abstract classes remain useful
8	`equals()` with a direct cast	Check `this == o` and `instanceof` (type + null) before casting
8	Wrappers via `new Integer(10)` etc.	Those constructors are deprecated for removal; use `valueOf` or autoboxing
8	Integer implements `Comparator<Integer>`	It implements `Comparable<Integer>`
9	OOM/StackOverflow are siblings of VirtualMachineError	They are its **subclasses**; FileNotFoundException is a subclass of IOException
9	Errors cannot be handled	They can be caught but usually shouldn't be
9	`throws` lets us ignore checked exceptions	It delegates them to the caller; nothing is ignored
9	finally always executes	Except on `System.exit`, a JVM crash/kill, or a try that never finishes

A.3 How to Revise This Book

When	What to do
First pass (1–2 weeks)	Read one phase per day; type out and run every code block in JShell or your IDE; note anything surprising
Daily (15 minutes)	Read one phase's **Quick Revision** and **Predict the Output** sections without looking at the answers
Weekly	Re-answer every **Interview Questions** block aloud, in under 60 seconds each, as you would in an interview
Before an interview	Phase 15's rapid-fire table, this corrections log, the Big-O tables (Phase 14), the `equals`/`hashCode` and HashMap internals (Phases 8 and 14), and the concurrency basics (Phase 11)
To go deeper	Spring Boot (DI, REST, JPA), testing (JUnit 5, Mockito, REST Assured), SQL and JDBC, and system design for backend services

> **TIP**
> A good habit for every topic: **explain it, show it in code, state the pitfall.** For example: "HashMap uses hashCode to pick a bucket and equals to find the key; here's an immutable key class; mutating a key after insertion makes the entry unreachable."